

A HUMAN-LIKE AGENT FOR SCREEN-AWARE TASK AUTOMATION

H.I.G.A. Hasintha
(Student ID: IT22167378)

BSc (Hons) Degree in Information Technology
Specializing in Software Engineering

Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Sri Lanka

August 2025

A HUMAN-LIKE AGENT FOR SCREEN-AWARE TASK AUTOMATION

H.I.G.A. Hasintha
(Student ID: IT22167378)

Dissertation submitted in partial fulfillment of the requirements for the Bachelor of
Science (Hons) in Information Technology Specializing in Software Engineering

Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Sri Lanka

August 2025

DECLARATION

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the nonexclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Signature



.....

Date

24/04/2026

.....

The above candidate has carried out research for the bachelor's degree Dissertation under my supervision.

Signature of the supervisor

.....

Date

.....

ABSTRACT

This research presents the design and implementation of a Hardware-Based Human Interface Device (HID) Actuation module for the Human-Like Agent for Screen-Aware Task Automation system. The primary function of this component is to receive structured, step-by-step action plans generated by the LLM reasoning module and translate them into precise, low-level physical input signals that are executed on the target machine through an ESP32 microcontroller operating as a USB HID device.

The module employs a three-stage execution pipeline consisting of a Node.js-based API server that receives JSON-formatted commands, a serial communication bridge that transmits those commands to the ESP32 firmware, and an Arduino-based HID emulation layer that reproduces keyboard and mouse interactions indistinguishable from those of a real human user. This design enables the system to interact with graphical user interfaces in a non-intrusive and platform-agnostic manner.

Unlike conventional automation frameworks that depend on internal application access, accessibility APIs, or DOM structures, the proposed approach operates through external physical input emulation. This design allows interaction with black-box systems, legacy platforms, and secure environments where traditional automation methods are limited or infeasible.

The system is evaluated through functional test cases covering mouse movement, click actions, text entry, drag operations, and key press sequences. Results indicate reliable execution with a 100% success rate across all tested scenarios. By integrating hardware-based HID emulation with structured command processing and feedback acknowledgment mechanisms, the proposed module demonstrates a robust and extensible approach to physical GUI automation.

Keywords – Human Interface Device (HID), ESP32, GUI Automation, Hardware Emulation, Black-Box Testing

ACKNOWLEDGEMENT

I would like to express my deep gratitude for all the people who helped me to continue my 4th year research project. First, I greatly appreciate our Research Project supervisor, who provided the guidance and encouragement to continue this project successfully. I am also grateful to our co-supervisor for the valuable and constructive suggestions given during the planning of our research work.

I also acknowledge my Research Project leader and other group members who have been supportive in pursuing these goals. Finally, I would like to give my heartfelt gratitude to my parents and friends who helped me and provided unending inspiration.

TABLE OF CONTENTS

DECLARATION	3
ABSTRACT.....	4
ACKNOWLEDGEMENT	5
TABLE OF CONTENTS.....	6
LIST OF FIGURES	9
LIST OF TABLES	10
LIST OF ABBREVIATIONS.....	11
1. INTRODUCTION	12
1.1 General Introduction	12
1.2 Background Literature	13
1.3 Limitation in the Current Approach.....	14
1.4 Research Gap	15
2. RESEARCH PROBLEM.....	18
2.1 Problem Context	18
2.2 Problem Statement	18
2.3 Core Research Problem.....	19
3. RESEARCH OBJECTIVES	20
3.1 Main Objective.....	20
3.2 Specific Objective.....	20
4. METHODOLOGY	22
4.1 System Architecture.....	22
4.2 illustrates the high-level layered architecture system diagram of the proposed system.	25
<i>Figure 4. 2: High-level layered architecture diagram</i>	25
4.2 Specific Component Description	26
4.2.1 Purpose.....	26
4.2.2 Workflow Overview	27
4.2.3 Command Reception and Processing.....	29
4.2.4 Serial Communication	30
4.2.5 Firmware-Level HID Action Execution	30
4.2.6 Integration with the Orchestration Layer	30
4.2.7 Error Handling and Recovery	30
4.2.8 Drag and Complex Input Support	31
4.2.9 Performance and Evaluation	31

4.3 HID Execution and Command Processing Framework	31
4.3.1 Command Input Formulation.....	31
4.3.2 Command Routing Strategy.....	31
4.3.3 HID Report Generation.....	32
4.3.4 Structured ACK Protocol.....	32
4.3.5 Output Validation and Correction.....	32
4.3.6 Feedback-Driven Sequential Dispatch.....	32
4.3.7 Execution Performance Optimisation	32
4.4 Design System Diagrams.....	33
4.4.1 DFD Diagram.....	33
.....	33
5. IMPLEMENTATION AND TESTING	34
5.1 Implementation	34
5.1.1 HID Service Overview.....	34
5.1.2 ESP32 Firmware Integration.....	35
5.1.3 Command-to-HID Pipeline.....	36
5.1.4 Coordinate Mapping and Screen Calibration.....	40
5.1.5 Action Type Implementation	40
5.1.6 Hardware Selection and Operational Decision	41
5.2 Testing.....	41
5.2.1 Test Plan and Strategy	41
5.2.2 Test Case Design.....	42
6. RESULTS AND DISCUSSIONS.....	45
6.1 Results.....	45
6.1.1 Overall Execution Outcomes	45
6.1.2 Quantitative Performance Summary.....	46
6.1.3 Accuracy and Output Structure Quality.....	47
6.1.4 Robustness Under Edge Conditions.....	48
6.1.5 Comparative Behaviour Across Scenario Types	49
6.1.6 Practical Result Interpretation.....	49
6.2 Research Finding	50
6.3 Discussion.....	52
6.3.1 Interpretation Against Research Objectives.....	52
6.3.2 Effectiveness of the Proposed Approach	52

6.3.3 Implication for Human-Computer Interaction	53
6.3.4 Reliability-Latency Trade-off	53
6.3.5 Workflow Continuity as a Limitation and Opportunity	53
6.3.6 Trust, Safety and Explainability	54
6.3.7 Threats to Validity	54
6.3.8 Practical Deployment Implications	54
7. CONCLUSIONS	55
8. REFERENCES	57
9. APPENDICES	58
Appendix - A: Plagiarism Report	58

LIST OF FIGURES

Figure 4.1: Overview of system architecture diagram	23
Figure 4.2: High-level layered architecture diagram	25
Figure 4.3: HID actuation component diagram	27
Figure 4.4: HID execution workflow	29
Figure 4.5: DFD diagram for HID Actuation Module	33
Figure 5.1: HID API endpoint flow diagram	35
Figure 5.2: ESP32 serial client implementation flow	36
Figure 5.3: JSON command parsing and serial dispatch	39
Figure 5.4: Generated HID command output	41
Figure 6.1: Terminal output of HID command execution	46
Figure 6.2: Postman request payload for HID endpoint	46

LIST OF TABLES

Table 1.1: Limitations and solutions	16
Table 4.1: Technologies, techniques, architecture and algorithms used	24
Table 5.1: Verify mouse movement and click execution	43
Table 5.2: Verify missing target element handling	43
Table 5.3: Verify text entry via HID keyboard emulation	44
Table 5.4: Verify JSON command output correctness	44
Table 5.5: Verify full login sequence execution	45

LIST OF ABBREVIATIONS

Abbreviation	Description
HID	Human Interface Device
ESP32	Espressif Systems ESP32 Microcontroller
API	Application Programming Interface
GUI	Graphical User Interface
USB	Universal Serial Bus
JSON	JavaScript Object Notation
OCR	Optical Character Recognition
LLM	Large Language Model
RPA	Robotic Process Automation
DOM	Document Object Model
DFD	Data Flow Diagram
ACK	Acknowledgement
MCU	Microcontroller Unit

1. INTRODUCTION

1.1 General Introduction

The Screen-Aware Task Agent is an agentic artificial intelligence-based automation system designed to interact with graphical user interfaces (GUIs) in a human-like manner. The system is developed to support task automation in environments where conventional software integration techniques are limited or infeasible, such as legacy systems, secure platforms, or applications that do not expose APIs. Unlike traditional automation approaches that rely on direct programmatic access, the proposed system operates through external visual observation, interpreting interface states and executing actions via physical input emulation. This approach aligns with emerging trends in vision-based automation and black-box testing methodologies.

The system is based on a closed-loop workflow consisting of perception, planning, action, and evaluation. In the perception stage, an external camera captures the target screen, and computer vision techniques including optical OCR are applied to extract textual and structural information from the interface. The planning stage utilizes a Large Language Model (LLM) to transform user-provided natural language instructions into structured, step-by-step action sequences. These actions are then executed in the action stage through a hardware-based input mechanism — this is the component addressed in this dissertation.

This component, the HID Actuation Module, is responsible for the physical execution layer of the system. Using an ESP32-S3 microcontroller configured to operate as a USB Human Interface Device, the module receives structured JSON command payloads from a Node.js API server and translates them into real mouse and keyboard input signals delivered to the target machine. From the perspective of the target operating system, these inputs are indistinguishable from those of a real human user. Finally, the evaluation stage analyzes visual feedback and execution logs to determine task completion and guide subsequent actions.

To ensure scalability and modularity, the system is implemented using a microservices-based architecture. A FastAPI-based backend functions as the central orchestrator, managing communication between independent services. The vision service is responsible for screen perception and data extraction, the LLM service handles instruction interpretation and action planning, and the HID Actuation Service manages physical input execution through the ESP32.

This architectural approach follows established principles of modular and distributed system design for scalable software engineering. Additionally, a React-based frontend provides a user interface for task input and system monitoring.

The proposed approach demonstrates how the integration of artificial intelligence, computer vision, large language models, and hardware-based input emulation can facilitate flexible and non-intrusive automation. By incorporating context-aware reasoning and visual feedback mechanisms, the system can handle dynamic and unpredictable interface conditions more effectively than traditional rule-based automation methods. Although the system is evaluated in controlled environments, it lays the foundation for further research in autonomous task execution, adaptive interface interaction, and intelligent human-computer interaction systems.

1.2 Background Literature

Graphical User Interface automation has been studied for decades to improve software quality and reduce repetitive manual interaction. Classical approaches such as WebDriver-based automation and rule-driven Robotic Process Automation (RPA) pipelines perform effectively when tools have access to internal application structures, including DOM metadata, application programming interfaces (APIs), or accessibility trees. In stable environments, these methods provide high precision and execution speed. However, their robustness decreases significantly in black-box settings where internal hooks are unavailable, unstable, or intentionally restricted.

To address the limitations of internal-access dependency, researchers have explored vision-based approaches for interface understanding. Optical Character Recognition techniques are commonly used to extract textual information from screenshots or camera-captured frames, with systems such as Tesseract OCR serving as widely adopted baselines. Furthermore, datasets and studies on interface structure, such as the Rico dataset, have supported data-driven modeling of UI layouts. Despite these advancements, vision-based systems primarily focus on perception and often lack robust physical execution capabilities for real-world environments.

Hardware-assisted interaction provides a compelling alternative solution for automation in restricted environments. Human Interface Device (HID) based techniques utilize microcontrollers, such as the ESP32, to emulate keyboard and mouse inputs without requiring any software-level integration on the host system. This approach is particularly useful in secure or legacy

environments because it operates entirely at the hardware layer. The USB HID standard, maintained by the USB Implementers Forum, defines a universal protocol by which input devices communicate with host systems, and this standard is leveraged by the proposed module to achieve transparent, cross-platform interaction.

Recent literature on microcontroller-based HID emulation has demonstrated the viability of using ESP32-class devices for this purpose. The ESP32-S3, in particular, supports native USB, enabling it to act as a composite HID device supporting simultaneous keyboard and mouse emulation. Prior works in security research and accessibility tools have explored USB HID emulation for keystroke injection and mouse control, demonstrating feasibility at the hardware and firmware level. However, the integration of such hardware with a fully agentic, LLM-driven planning pipeline for general-purpose GUI automation remains underexplored in academic research.

Overall, the literature demonstrates significant advancements in the individual domains of perception, reasoning, and execution. Nevertheless, the integration of a hardware-based physical execution layer into a unified, closed-loop automation framework for real-world black-box environments remains limited. This limitation highlights the need for a robust HID actuation module capable of receiving structured commands and translating them into reliable physical interactions within such a cohesive system.

1.3 Limitation in the Current Approach

Although the proposed HID Actuation module demonstrates the feasibility of hardware-driven, non-intrusive GUI automation, several limitations remain in the current implementation.

First, the execution layer is subject to latency inherent in the serial communication pathway between the Node.js API server and the ESP32 microcontroller. While mouse click and keyboard press actions can be dispatched within milliseconds, operations requiring coordinated movement, such as drag-and-drop sequences, introduce measurable delays on the order of one to two seconds. This latency is primarily a consequence of the USB serial protocol overhead and the line-by-line command dispatch approach currently employed.

Second, the reliability of the communication channel is partially dependent on the stability of the USB serial connection between the host machine and the ESP32 device. Environmental

factors such as USB port quality, cable integrity, and host-side driver behavior may introduce intermittent disconnections or data corruption that adversely affect command delivery and execution continuity.

Third, the current implementation operates on a synchronous command-response model. Each HID command is dispatched and an acknowledgement is awaited before the next command is issued. While this ensures sequential correctness, it limits throughput and may be suboptimal for scenarios requiring rapid or concurrent input events, such as complex drag-and-drop interactions or multi-key chord inputs.

Fourth, the coordinate system used for mouse movement is tied to the screen resolution of the environment in which the system is tested. If the target display resolution changes, or if the system is deployed across monitors with different scaling factors, the coordinate mapping must be recalibrated. The current implementation does not include automatic resolution detection or coordinate normalization mechanisms.

Fifth, the evaluation of the system has been conducted within controlled environments and a limited set of application scenarios. As a result, the generalizability of the HID actuation approach across diverse operating systems, screen resolutions, input handling behaviors, and real-world deployment conditions has not been fully validated.

1.4 Research Gap

While existing automation frameworks have explored both software-level scripting and hardware-level input emulation, several critical gaps remain in the effective application of hardware HID devices for agentic, AI-driven GUI automation in real-world black-box environments.

One major limitation is the absence of a structured command protocol bridging high-level AI-generated action plans and low-level physical HID execution. Many existing HID emulation tools operate as standalone utilities that accept predefined scripts or direct user commands, rather than acting as downstream execution consumers within a broader intelligent automation pipeline. The absence of a standardized, machine-readable command format — such as a JSON-based schema — makes integration with LLM-generated structured outputs difficult and error-prone.

Another critical challenge is the lack of feedback and acknowledgement mechanisms in hardware-level execution layers. In practical automation scenarios, each physical action must be confirmed as executed before the next instruction is dispatched. However, many existing HID emulation implementations do not return acknowledgement signals, leaving the orchestration layer unable to determine whether an action succeeded or failed. This limits reliability in iterative, closed-loop automation workflows.

Furthermore, current HID emulation approaches are typically developed for narrow, predefined use cases such as keystroke injection or simple macro playback. They are not designed to handle a diverse range of action types — including mouse movement, left and right clicks, text entry, key press sequences, and drag operations — within a single cohesive execution framework. The absence of a generalized, action-type-aware execution layer limits the breadth of tasks that can be automated through hardware-based methods.

In addition, a significant gap exists between the latency characteristics of hardware-based execution and the real-time responsiveness expected by agentic systems. While software-based automation tools can execute actions in sub-millisecond timeframes, hardware HID devices must account for USB communication overhead, firmware processing time, and physical device response delays. These latency considerations are not adequately addressed in existing research, particularly in the context of screen-aware, perception-driven automation.

Therefore, a clear research gap exists in the development of a structured, feedback-driven HID execution framework that can receive machine-readable action commands, reliably translate them into physical input signals across diverse action types, return acknowledgement status to the orchestration layer, and operate within the latency constraints of a real-time closed-loop automation system.

Aspect	Current Limitation	Impact	Proposed Direction
Command Format	No standardized protocol for HID input	Incompatible with AI pipelines	Use JSON-based command schema
Feedback	No acknowledgement from hardware	Unreliable closed-loop execution	Implement ACK protocol per command

Action Coverage	Limited to simple keystrokes or fixed macros	Cannot handle diverse GUI actions	Support mouse, keyboard, drag, and key events
Latency	Not optimized for agentic loops	Delays disrupt real-time control	Optimize serial dispatch and firmware response

Table 1.1: Limitations and solutions

The proposed HID Actuation Module addresses this gap by introducing a structured, ESP32-based execution framework that accepts JSON-formatted action commands from the orchestration layer, translates them into precise physical HID signals, and returns acknowledgement status after each command execution. This approach is specifically designed to operate within a real-world, agentic automation pipeline, thereby improving the reliability, generality, and practical applicability of hardware-based GUI automation systems.

2. RESEARCH PROBLEM

2.1 Problem Context

Large Language Models have recently emerged as powerful tools for interpreting natural language instructions and generating task-oriented action sequences. Their ability to understand user intent, process contextual information, and produce structured responses makes them highly suitable for automation scenarios where tasks are defined using human-readable instructions. However, the ability of an AI system to plan and reason about GUI actions is only meaningful if those planned actions can be reliably translated into physical, real-world input signals that reach the target machine.

In the proposed Screen-Aware Task Agent, the LLM and prompt engineering module is responsible for producing structured action plans. However, these plans remain purely digital artifacts unless a dedicated execution layer exists to bridge the gap between digital instruction and physical action. In environments where the target system cannot be accessed programmatically — such as legacy platforms, secure kiosks, or black-box systems — this execution layer must operate externally and at the hardware level, emulating the physical behaviour of a human operator.

The HID Actuation Module addresses this need by acting as the physical hand of the system. It receives structured command payloads from the Node.js API server, communicates them to an ESP32 microcontroller over USB serial, and executes them as real mouse and keyboard input signals. This positions the HID module as a critical enabling component without which the broader agentic system cannot perform any meaningful interaction with the target environment.

2.2 Problem Statement

Despite the availability of microcontroller-based HID emulation technology, applying it effectively within an AI-driven, closed-loop automation system introduces several critical challenges that are not addressed by existing implementations.

One major issue is that existing HID emulation tools are not designed to consume structured, machine-readable command formats generated by AI planning systems. Most available solutions accept hardcoded scripts or direct serial commands in proprietary formats, making them incompatible with JSON-based structured action plans produced by LLM pipelines. This creates a translation gap between what the AI system produces and what the hardware can consume.

In addition, the absence of a reliable acknowledgement mechanism makes it difficult to verify that each command was successfully executed before the next is dispatched. In complex multi-step workflows, a failed or missed action can cause subsequent steps to target incorrect interface states, resulting in cascading errors. Without an ACK protocol at the hardware level, the orchestration system has no ground truth about execution status.

Furthermore, most existing HID emulation frameworks are limited to a single type of input, either keyboard or mouse, and do not provide a unified command interface that supports the full range of GUI interaction types required by a general-purpose automation agent. Actions such as smooth mouse movement, drag-and-drop, text entry, modifier key combinations, and multi-step click sequences all require distinct command handling logic that must be consolidated into a single, robust execution framework.

Another important limitation is the latency introduced by the serial communication pathway. If the ESP32 firmware and the serial dispatch logic are not optimised for throughput, cumulative delays across a multi-step action plan can significantly degrade end-to-end automation performance, undermining the practical utility of the system.

2.3 Core Research Problem

Therefore, the core research problem addressed in this study is the absence of a robust, structured, and feedback-driven hardware HID execution framework capable of reliably bridging the gap between AI-generated structured action plans and real-world physical GUI interaction. Such a framework must be capable of receiving JSON-formatted action commands covering a comprehensive set of interaction types, translating those commands into accurate and timely HID signals through an ESP32-based microcontroller, providing per-command acknowledgement feedback to the orchestration layer, and operating within the latency and reliability constraints demanded by a real-time agentic automation system.

Addressing this problem is essential for enabling reliable and scalable automation in complex environments where systems must operate with minimal assumptions, interact transparently with any target platform, and perform physical interactions that are indistinguishable from those of a real human user. The development of such a framework represents a critical step toward achieving practical, human-like automation in real-world applications.

3. RESEARCH OBJECTIVES

3.1 Main Objective

The primary objective of this study is to design and implement a robust, hardware-based HID Actuation Module capable of receiving structured, machine-readable action commands from an AI orchestration layer and translating them into precise, reliable physical input signals delivered to a target computer through an ESP32 microcontroller operating as a USB Human Interface Device. The module is designed to bridge the gap between AI-generated high-level action plans and real-world physical GUI interaction, enabling the broader Screen-Aware Task Agent system to perform adaptive, human-like automation in black-box environments.

Furthermore, the proposed solution aims to support a comprehensive range of GUI interaction types — including mouse movement, left and right clicks, drag-and-drop operations, text entry, and keyboard key press sequences — within a unified, feedback-driven execution framework. By ensuring per-command acknowledgement and reliable serial communication, the module enables the orchestration layer to maintain ground truth about execution status and adapt its planning accordingly. Ultimately, the objective is to create a physical execution layer that is platform-independent, transparent to the target system, and operationally consistent across diverse automation scenarios.

3.2 Specific Objective

The specific objectives of this study are to design a JSON-based command protocol that standardises the representation of GUI action commands — including mouse movement, click, drag, type, and key press operations — in a machine-readable format that can be directly consumed by the HID execution firmware without additional transformation.

In addition, the study aims to implement a Node.js-based API server that exposes RESTful endpoints for receiving structured action commands from the orchestration layer, validates incoming payloads, and dispatches them to the ESP32 microcontroller over a USB serial communication channel.

Furthermore, the research focuses on developing Arduino-based firmware for the ESP32-S3 microcontroller that parses incoming JSON command strings, maps them to the appropriate HID input library calls, executes the corresponding physical mouse or keyboard actions, and returns an acknowledgement response to confirm successful execution.

Another key objective is to ensure that the HID actuation layer operates in a fully platform-agnostic manner by relying exclusively on the USB HID standard. By eliminating any dependency on host-side software installation, accessibility APIs, or internal application hooks, the module is designed to interact with any target system that supports USB input devices, including legacy, secure, and cross-platform environments.

Additionally, the study aims to evaluate the functional reliability and latency characteristics of the HID execution pipeline across a range of test scenarios, including standard workflows, edge cases involving missing targets, and multi-step sequential interactions, in order to assess the practical suitability of the module for closed-loop agentic automation.

Finally, the system is designed to produce structured, traceable output artefacts — including command logs, acknowledgement records, and execution timing data — to support debugging, audit, and integration with the broader observability infrastructure of the Screen-Aware Task Agent system.

4. METHODOLOGY

The methodology for the HID Actuation Module in the Human-Like Agent for Screen-Aware Task Automation project is designed to physically execute structured UI action commands received from the LLM reasoning layer. The process involves multiple integrated steps: command reception, JSON parsing, serial dispatch, firmware-level HID emulation, and acknowledgement feedback.

4.1 System Architecture

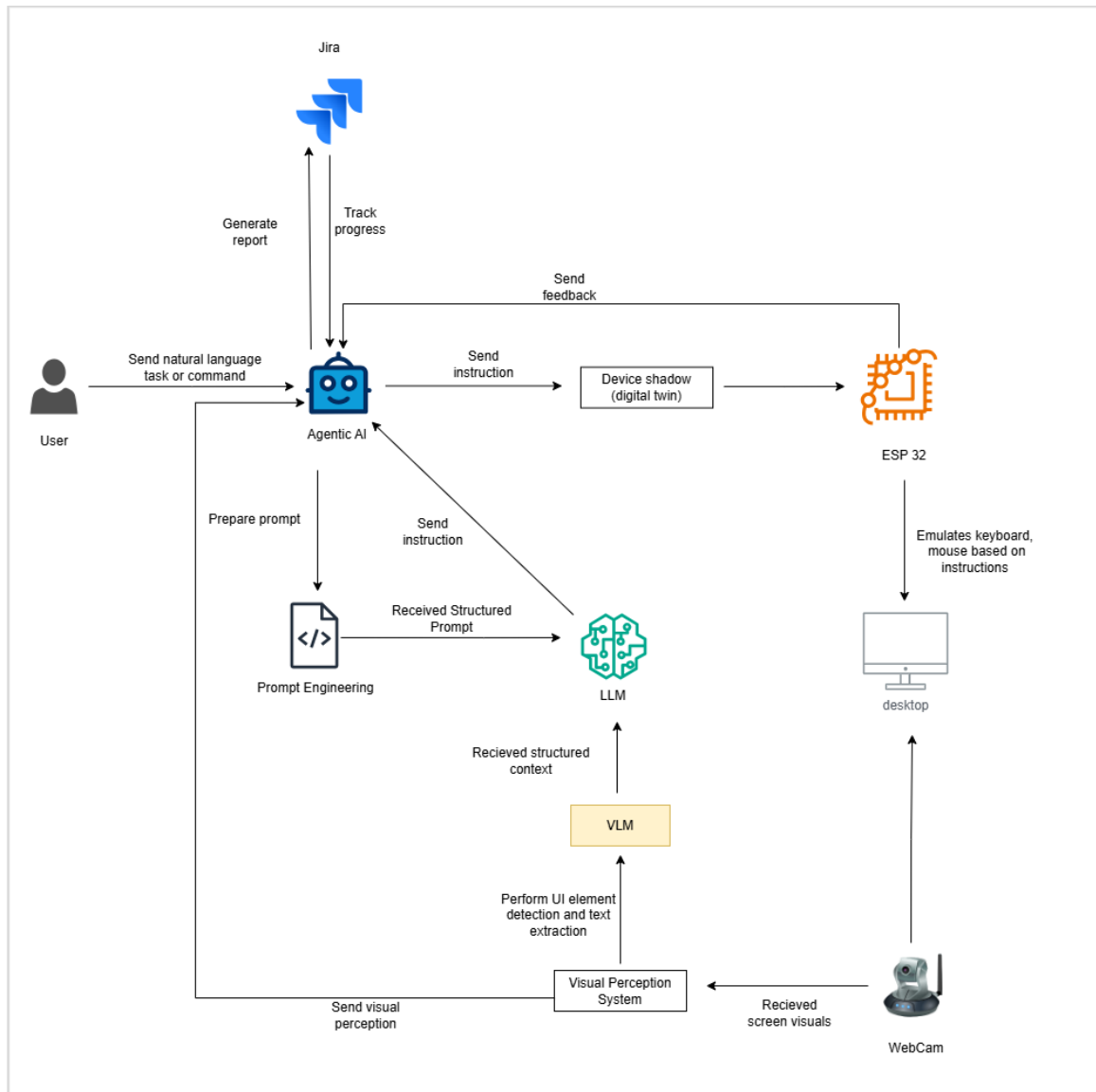


Figure 4. 1: Overview of system architecture diagram

Figure 4.1 presents the end-to-end architecture of the Screen-Aware Task Agent as a closed-loop, agentic automation system designed for black-box GUI interaction. The architecture is intentionally built around external observation and hardware-level interaction so that automation can be performed even when internal application APIs, source code, or DOM hooks are unavailable. At a high level, the system follows a continuous perceive-plan-act-evaluate cycle, where each execution step is validated against the latest visual state before the next action is issued.

The workflow starts at the user interaction layer, where a tester provides a high-level natural-language objective. This instruction is received by the Agentic AI controller, which delegates interpretation to the LLM and prompt engineering pipeline. The LLM produces structured, machine-readable action steps in JSON format. These steps are forwarded to the HID Actuation Service — the component addressed in this dissertation — which is responsible for all physical execution on the target machine.

The HID Actuation Service receives structured command payloads from the Agentic AI core via HTTP REST and forwards them over USB serial to the ESP32-S3 microcontroller. The ESP32 firmware parses each command and executes the corresponding HID input operation — mouse movement, click, text entry, or key press — directly on the target computer. The target machine receives these inputs as if they originated from a real hardware peripheral, ensuring complete transparency and platform independence.

After each action is executed, the ESP32 firmware returns an acknowledgement response over the serial channel, confirming the command was received and processed. This ACK is relayed back through the Node.js service to the orchestration layer, which uses it to gate dispatch of the next command. This feedback mechanism ensures sequential correctness and prevents command loss in multi-step execution flows.

The architecture also includes observability integrations. Execution traces, command logs, timing data, and completion outcomes are collected during each run to support traceability and debugging. These logs can be synchronised with the broader project management and reporting infrastructure used by the group.

Summary of technologies, techniques, architecture and algorithms used for effective resource is shown in the table 4.1 below.

Category	Details
Technologies	Node.js (Express), ESP32-S3, Arduino USB HID Stack, USB Serial, JSON
Techniques	JSON command parsing, ACK-driven sequential dispatch, HID input mapping, serial communication
Algorithms	Action-type routing, coordinate mapping, retry-on-NAK logic, command queue management
Architectures	Microservices architecture, hardware-in-the-loop execution, closed-loop agentic control

Table 4.1: Technologies, techniques, architecture and algorithms used

4.2 illustrates the high-level layered architecture system diagram of the proposed system.

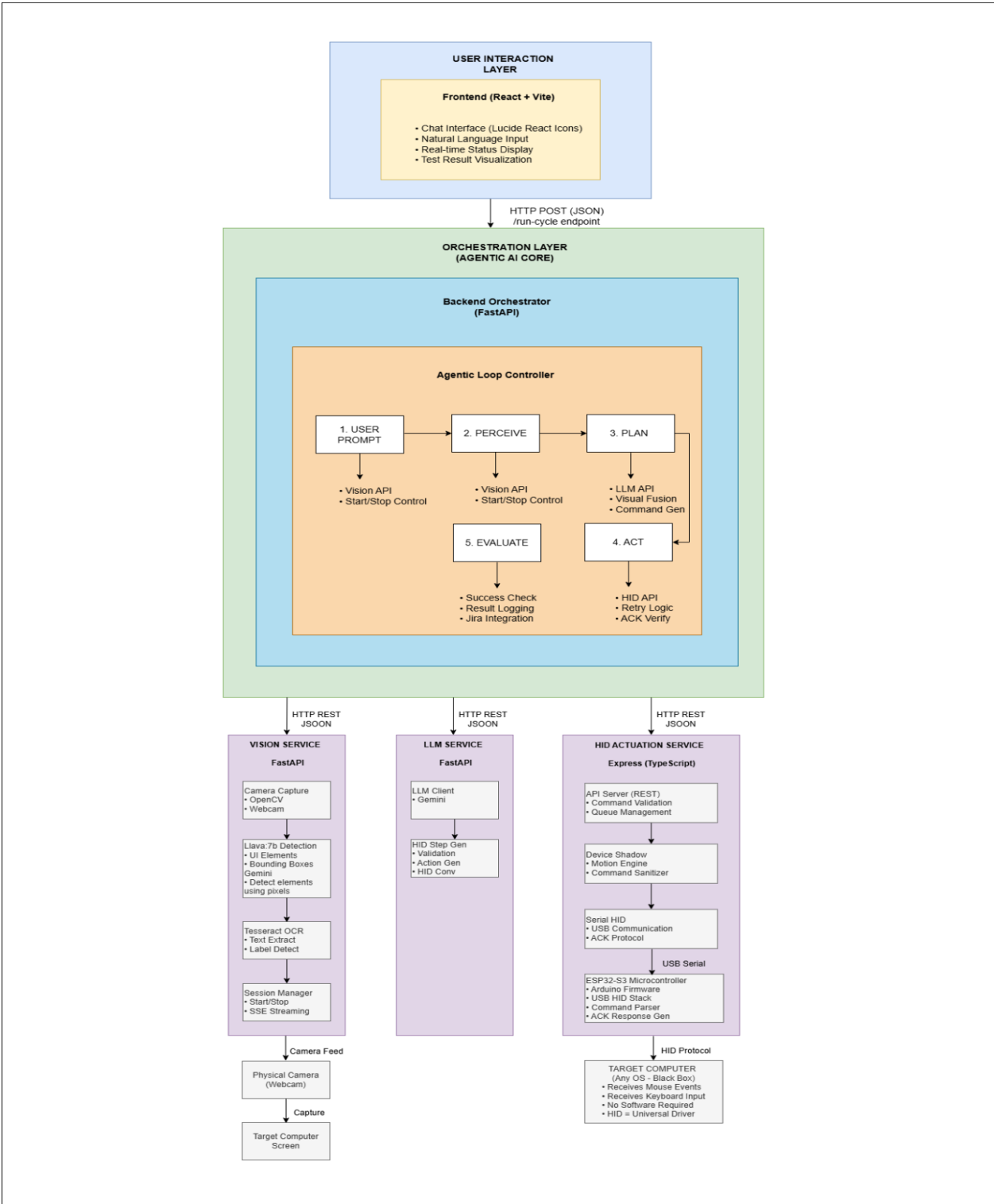


Figure 4. 2: High-level layered architecture diagram

4.2 Specific Component Description

The HID Actuation Module is the physical execution unit of the Screen-Aware Task Agent, responsible for converting structured action plans produced by the LLM reasoning layer into real hardware-level input signals delivered to the target machine. This component acts as the bridge between digital planning and physical interaction, ensuring that the system can execute precise GUI actions in a manner indistinguishable from a real human operator.

4.2.1 Purpose

The HID Actuation Module receives structured JSON command payloads from the orchestration layer and translates each command into the corresponding physical input event on the target machine. It supports a comprehensive set of GUI interaction types including mouse movement, left and right clicks, text entry via keyboard emulation, key press and release sequences, and drag-and-drop operations. The module also implements an acknowledgement protocol, returning a confirmation response after each successfully executed command, enabling the orchestration layer to maintain reliable control over multi-step execution flows.

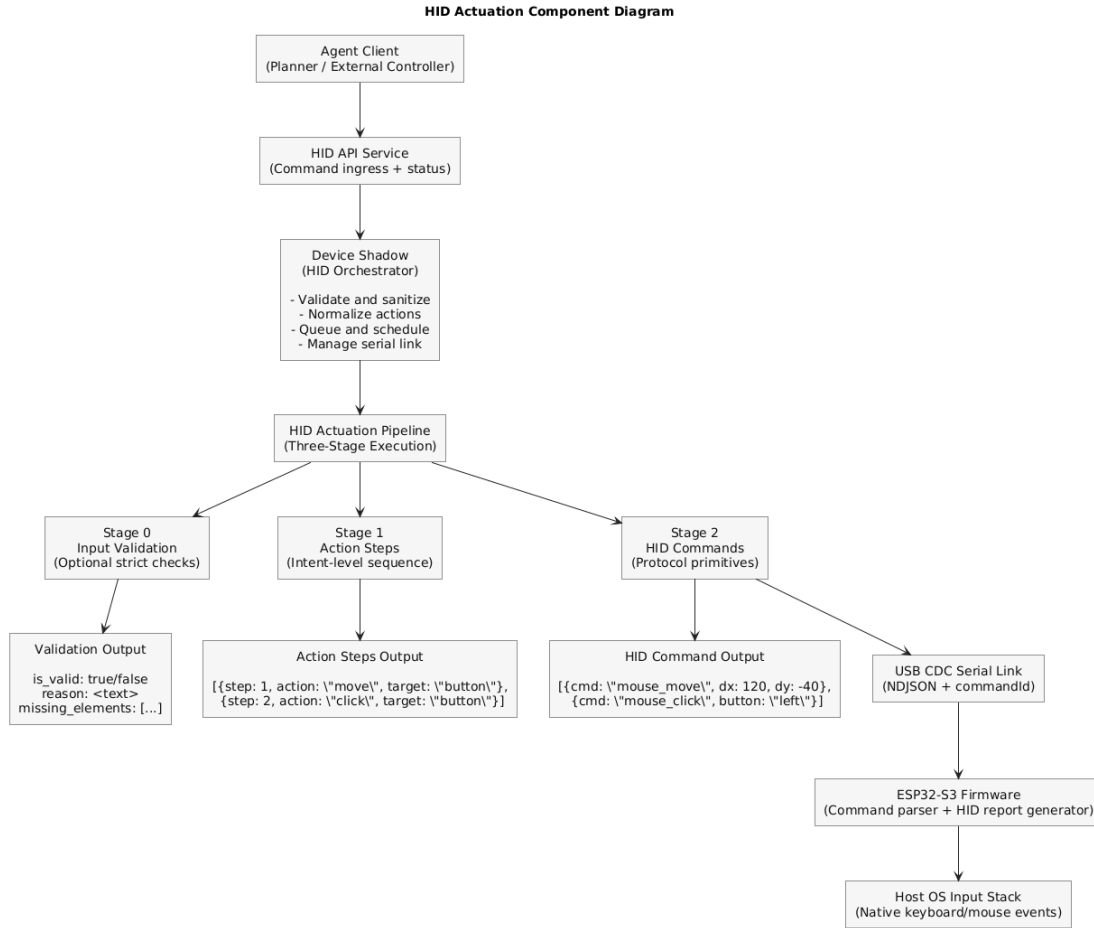


Figure 4.3: HID actuation component diagram

4.2.2 Workflow Overview

The module operates through a comprehensive pipeline that processes structured action commands into physical HID interactions:

Command Reception and Validation: The Node.js API server receives a JSON payload from the orchestration layer via HTTP POST. The payload is validated for schema correctness, including the presence of required fields such as command type, target coordinates, and optional parameters such as input text or key identifiers.

Serial Dispatch: The validated command is serialised and transmitted to the ESP32 microcontroller over the USB serial channel. Each command is sent as a single-line JSON string terminated with a newline character to ensure reliable framing at the firmware level.

Firmware Parsing and Action Mapping: The ESP32 firmware reads the incoming serial data, parses the JSON string, and routes the command to the appropriate HID action handler based on the command type field. Supported action types include `mouse_move`, `mouse_click`, `type_text`, `key_press`, and `drag`.

HID Execution: The firmware calls the appropriate USB HID library function to execute the physical input event. For mouse operations, absolute or relative coordinates are translated into HID report values. For keyboard operations, character sequences or key codes are mapped to HID usage IDs and transmitted as HID reports to the host operating system.

Acknowledgement Response: Upon successful execution of each command, the firmware returns an ACK response string over the serial channel. The Node.js server reads this response and forwards the status back to the orchestration layer, confirming that the command was executed and that the next command may be dispatched.

HID Execution Workflow Diagram

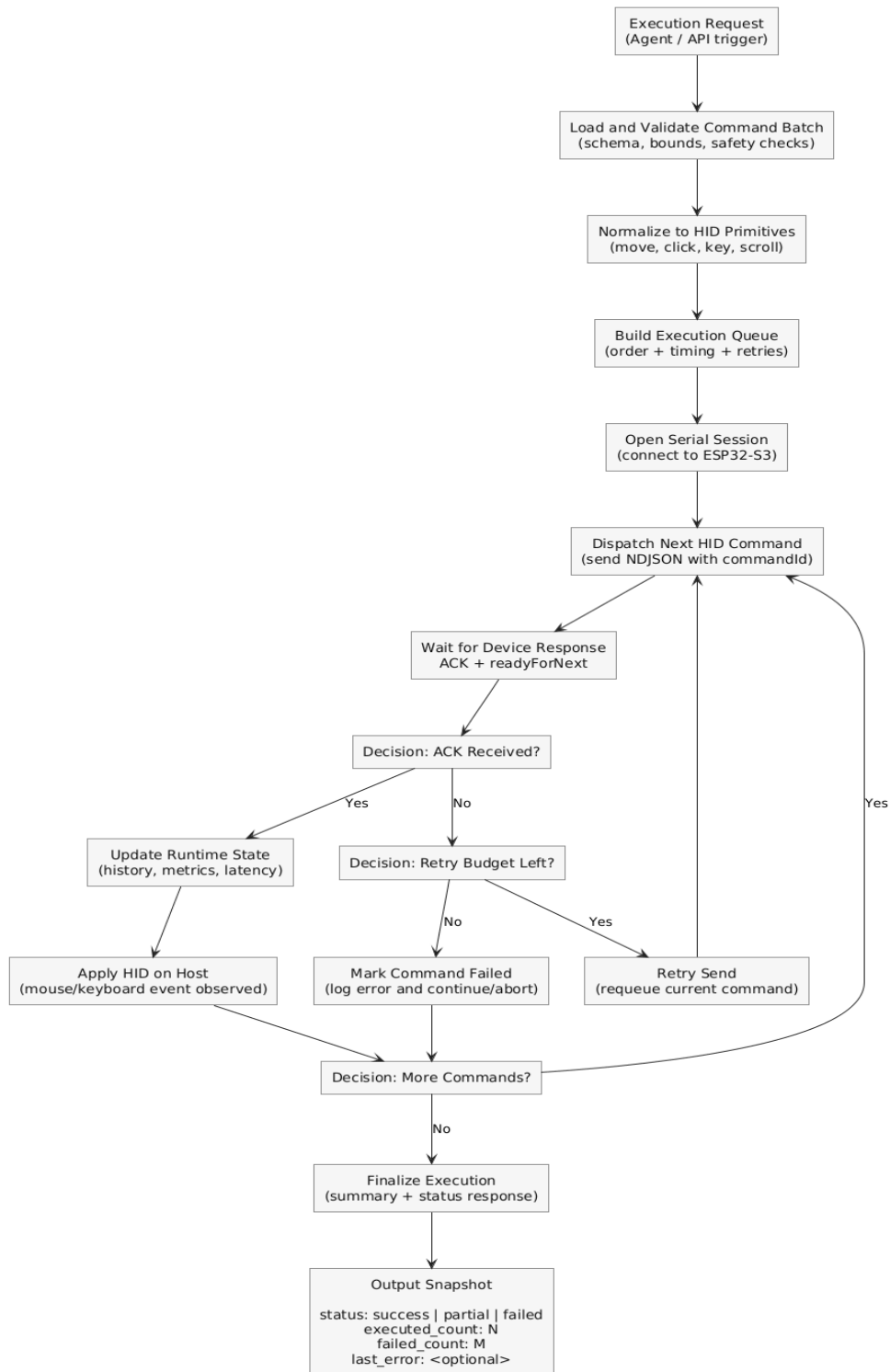


Figure 4.4: HID execution workflow

4.2.3 Command Reception and Processing

Structured JSON action commands are received at the Node.js API server through a RESTful HTTP endpoint. Each incoming request carries a command object specifying the action type and associated parameters such as target pixel coordinates, input text, or key identifiers. The server validates the command schema before dispatching it to the serial communication layer, ensuring that malformed or incomplete commands are rejected before reaching the firmware.

4.2.4 Serial Communication

Communication between the Node.js API server and the ESP32 microcontroller is implemented over USB serial using the serialport Node.js library. Commands are serialised as single-line JSON strings and transmitted at a fixed baud rate. The newline-delimited framing protocol ensures that the ESP32 firmware can reliably identify complete command boundaries even under conditions of partial data delivery. The server maintains an open serial port connection throughout the operational lifetime of the service.

4.2.5 Firmware-Level HID Action Execution

The ESP32-S3 firmware, written in the Arduino framework, uses the USB HID stack provided by the ESP32 Arduino core to present the microcontroller as a composite USB HID device supporting both keyboard and mouse functionality. Upon receipt of a serial command, the firmware deserialises the JSON payload using the ArduinoJson library, identifies the command type, and invokes the corresponding HID report function. Mouse movements are issued as absolute position reports. Clicks are reported as button press and release events. Text entry iterates over each character and maps it to the appropriate HID keyboard usage ID.

4.2.6 Integration with the Orchestration Layer

The HID Actuation Module integrates directly with the Agentic AI core through its REST API endpoint. The orchestration layer dispatches one command at a time, waits for the ACK response, and only then proceeds to the next action in the plan. This synchronous dispatch model ensures that the physical state of the target machine is consistent with the assumed state at each point in the action sequence, reducing the risk of misaligned interactions in multi-step workflows.

4.2.7 Error Handling and Recovery

The module implements basic error handling at both the API and firmware levels. If the ESP32 returns a NAK or error response, the API server logs the failure and returns an error status to the orchestration layer, which can then decide whether to retry the command or trigger a re-

planning cycle. At the firmware level, if JSON parsing fails due to a malformed payload, the ESP32 returns a structured error message rather than attempting partial execution, preventing unintended physical actions.

4.2.8 Drag and Complex Input Support

For drag-and-drop operations, the firmware implements a multi-step sequence: move the mouse to the source coordinate, press and hold the left mouse button, move to the target coordinate in incremental steps to simulate natural movement, and then release the button. This approach reproduces the physical characteristic of a human drag operation and is compatible with graphical applications that rely on continuous motion events during drag interactions.

4.2.9 Performance and Evaluation

Performance of the HID Actuation Module is evaluated across multiple dimensions: execution latency per action type, success rate of command execution, correctness of physical input delivery, and reliability of the ACK protocol under repeated execution. These metrics collectively assess the module's effectiveness as the physical execution backbone of the broader agentic automation system.

4.3 HID Execution and Command Processing Framework

This section presents the methodology used for the HID Actuation Module. The objective is to translate structured action commands into reliable, low-latency physical input signals while maintaining sequential correctness through an acknowledgement-driven communication protocol.

4.3.1 Command Input Formulation

Incoming action commands are expressed as JSON objects specifying a command type identifier and a set of typed parameters appropriate to that action. The command type field acts as a dispatch key, routing each command to the correct handler in both the Node.js server and the ESP32 firmware. Parameter validation at the server level ensures that only well-formed commands reach the hardware.

4.3.2 Command Routing Strategy

The firmware implements a switch-case dispatch structure that routes each parsed command type to a dedicated handler function. This design separates the concerns of command

parsing, action execution, and response generation, ensuring that the addition of new command types does not require modification of existing handler logic.

4.3.3 HID Report Generation

Each handler function constructs the appropriate HID report using the USB HID stack. For mouse actions, reports include X and Y coordinate values and button state flags. For keyboard actions, reports include the key usage ID, modifier flags, and a subsequent null report to represent key release. The USB HID stack serialises these reports into the standard HID report format and transmits them to the host OS.

4.3.4 Structured ACK Protocol

Upon completion of each command, the firmware transmits a structured acknowledgement response over the serial channel. The response indicates the command identifier, execution status (ACK or NAK), and an optional timing field recording the duration of the execution. The Node.js server reads this response, parses it, and returns the status to the orchestration layer through the API response.

4.3.5 Output Validation and Correction

Before dispatching commands to the serial port, the API server performs schema validation on the incoming JSON payload. If a required field is missing, an out-of-range coordinate is detected, or an unsupported command type is specified, the server returns a 400 error response to the orchestration layer without transmitting anything to the hardware, preventing unintended input events.

4.3.6 Feedback-Driven Sequential Dispatch

The orchestration layer dispatches commands one at a time in strict sequence. After each command is dispatched, it waits for the ACK response before proceeding to the next command. This feedback-driven dispatch model ensures that the physical execution state of the target machine is always consistent with the assumed state at each step of the action plan, preserving the integrity of multi-step automation workflows.

4.3.7 Execution Performance Optimisation

The serial baud rate is configured at a value that balances throughput and reliability. The firmware minimises processing time per command by pre-allocating JSON document buffers and avoiding dynamic memory allocation during command handling. For multi-step sequences such

as drag operations, intermediate movement steps are batched within the firmware to reduce serial round-trip overhead.

4.4 Design System Diagrams

System design diagrams were developed to identify the essential components and organise the implementation of the proposed model. Figure 4.5 illustrates the DFD diagram that helped to develop this research component.

4.4.1 DFD Diagram

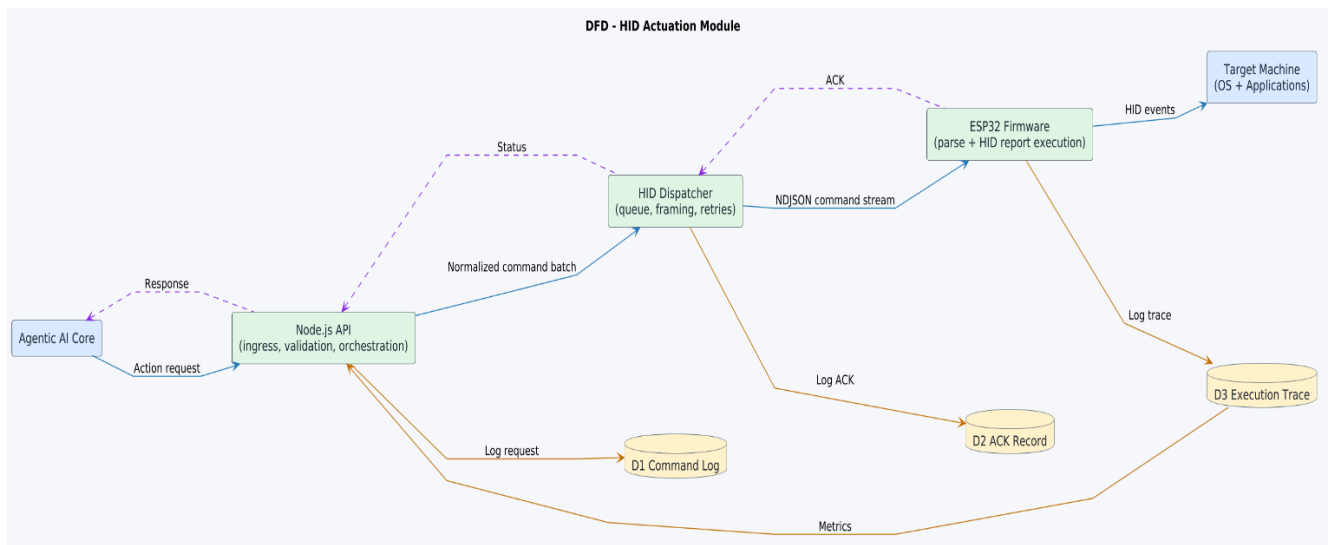


Figure 4.5: DFD diagram for HID Actuation Module

5. IMPLEMENTATION AND TESTING

5.1 Implementation

The implementation focuses on a structured, hardware-in-the-loop execution pipeline that translates JSON-formatted action commands into physical HID input signals delivered to the target machine through an ESP32-S3 microcontroller. The HID module is implemented as a two-tier system: a Node.js API server that exposes RESTful endpoints for command reception and serial dispatch, and an Arduino-based firmware running on the ESP32-S3 that handles HID report generation and acknowledgement. In the final operational setup, the ESP32-S3 is used as the sole execution hardware due to its native USB support and the stability of its HID emulation stack in iterative, closed-loop use. The implementation includes a JSON command schema that standardises the representation of all supported action types, a serial framing protocol that ensures reliable data delivery, and an ACK mechanism that confirms command execution before the next dispatch. This implementation supports a continuous plan-act-acknowledge loop for real-world black-box GUI automation.

5.1.1 HID Service Overview

The HID microservice serves as the physical execution component of the system and is responsible for converting structured action commands into USB HID input signals. It is implemented as a Node.js Express service that exposes endpoints for command dispatch, health checking, and runtime status monitoring. When a command is received, the service first validates the request schema, then serialises the command as a JSON string and transmits it over the USB serial port to the ESP32-S3 firmware. The service then awaits an acknowledgement response from the firmware before returning the execution status to the caller. This microservice-based design allows the HID component to operate independently while remaining tightly integrated with the LLM planning and visual perception layers of the broader system.

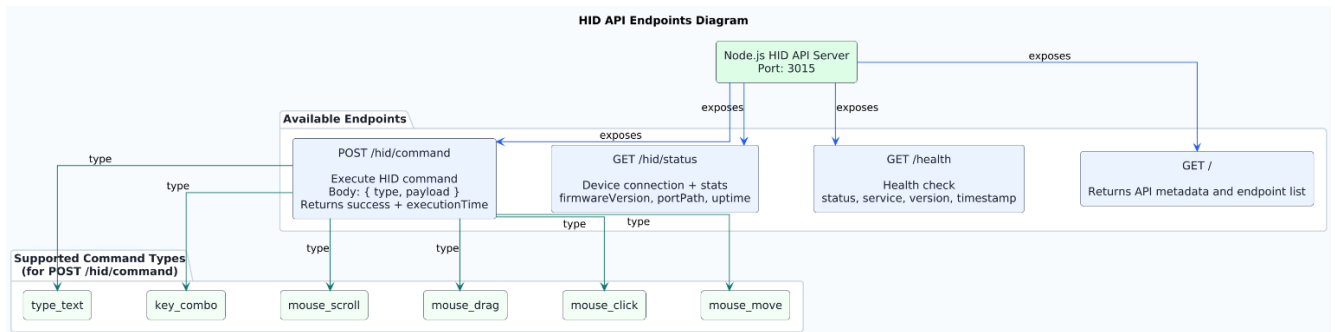


Figure 5.1: HID API endpoint flow diagram

5.1.2 ESP32 Firmware Integration

The ESP32-S3 firmware is implemented using the Arduino framework and the native USB HID library provided by the Espressif Arduino core. The firmware initialises the USB HID device on startup, presenting itself to the host operating system as a composite device with both keyboard and mouse capabilities. It then enters a loop that continuously reads incoming serial data, buffers characters until a complete JSON line is received, and processes each command through the action dispatch pipeline. The ArduinoJson library is used for lightweight, in-place JSON deserialisation without dynamic memory allocation. The firmware uses the ESP32 USB HID API to construct and send HID reports for each action type.

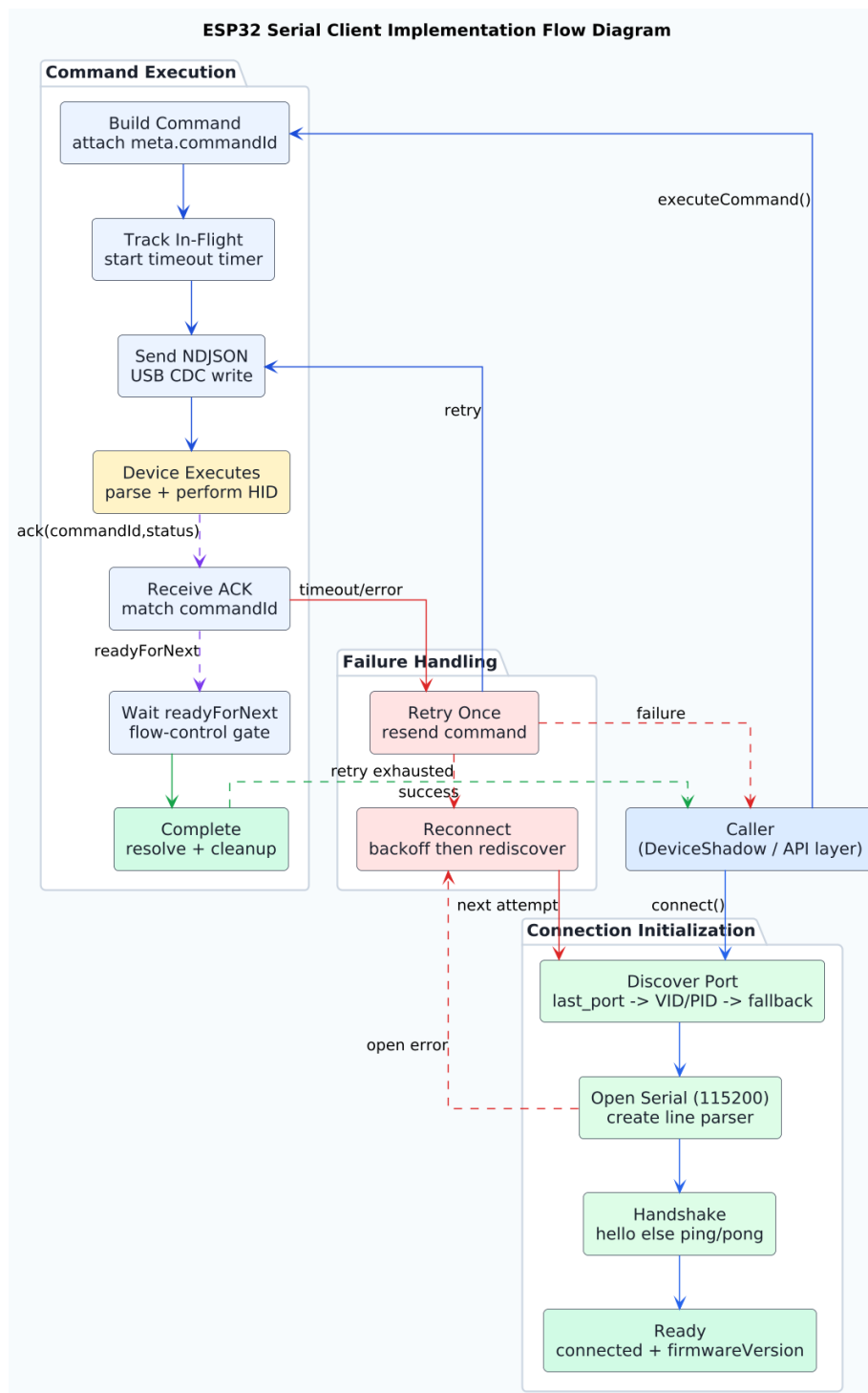


Figure 5.2: ESP32 serial client implementation flow

5.1.3 Command-to-HID Pipeline

The command-to-HID pipeline converts structured JSON action commands into physical HID input events. The process starts when the Node.js service receives an HTTP POST request containing a command object. The service validates the payload, serialises it as a newline-terminated JSON string, and writes it to the serial port. The ESP32 firmware reads the string, parses the JSON, and routes the command to the appropriate handler. Each handler constructs the correct HID report — mouse move, click, type, key press, or drag — and transmits it via USB. Upon completion, the firmware writes an ACK string to serial, which the Node.js service reads and returns to the caller. This pipeline is a key implementation component because it transforms abstract action objects into deterministic, physically reproducible input events.

```
void loop() {
  // Send periodic heartbeat to maintain connection health
  if (millis() - lastHeartbeat > HEARTBEAT_INTERVAL) {
    sendHello(); // Re-send hello as heartbeat
    lastHeartbeat = millis();
  }

  // Check for incoming commands
  if (Serial.available()) {
    String cmdLine = Serial.readStringUntil('\n');
    cmdLine.trim();

    if (cmdLine.length() == 0) {
      // empty line, ignore
    }
    else {
      // Only attempt to parse lines that look like JSON objects to avoid
      // treating stray boot/test output as commands (e.g., "tick: ...").
      if (cmdLine.charAt(0) == '{') {
        // Safety: limit maximum length to avoid memory exhaustion
        if (cmdLine.length() <= 1024) {
          processCommand(cmdLine);
        } else {
          sendError("line_too_long", "Input line exceeds allowed length");
        }
      } else {
        // Ignore non-JSON lines silently to keep host logs clean
      }
    }
  }

  // Small delay to prevent CPU spinning
  delay(1);
}
```

```

void processCommand(const String& cmdLine) {
    // Clear previous document
    jsonDoc.clear();

    // Parse JSON
    DeserializationError error = deserializeJson(jsonDoc, cmdLine);

    if (error) {
        sendError("invalid_json", error.c_str());
        return;
    }

    // Extract command ID from meta field (for ACK system)
    if (jsonDoc.containsKey("meta") && jsonDoc["meta"].containsKey("commandId")) {
        const char* cmdId = jsonDoc["meta"]["commandId"];
        if (cmdId) {
            currentCommandId = String(cmdId);
        }
    } else {
        currentCommandId = "";
    }

    // Check if this is a control message (type field) or command (cmd field)
    const char* msgType = jsonDoc["type"];
    const char* cmd = jsonDoc["cmd"];

    // Handle control messages (ping/pong, etc.)
    if (msgType) {
        if (strcmp(msgType, MSG_PING) == 0) {
            sendPong();
            return;
        }
        // Unknown message type - ignore silently
        return;
    }

    // Handle commands
    if (!cmd) {
        sendError("missing_cmd", "No 'cmd' field in JSON");
        return;
    }
}

```

```

// Route to appropriate handler
if (strcmp(cmd, CMD_MOUSE_MOVE) == 0) {
    handleMouseMove();
}
else if (strcmp(cmd, CMD_MOUSE_CLICK) == 0) {
    handleMouseClicked();
}
else if (strcmp(cmd, CMD_MOUSE_DOWN) == 0) {
    handleMouseDown();
}
else if (strcmp(cmd, CMD_MOUSE_UP) == 0) {
    handleMouseUp();
}
else if (strcmp(cmd, CMD_MOUSE_SCROLL) == 0) {
    handleMouseScroll();
}
else if (strcmp(cmd, CMD_KEY_PRESS) == 0) {
    handleKeyPress();
}
else if (strcmp(cmd, CMD_KEY_RELEASE) == 0) {
    handleKeyRelease();
}
else if (strcmp(cmd, CMD_TYPE_TEXT) == 0) {
    handleTypeText();
}
else if (strcmp(cmd, CMD_SYSTEM) == 0) {
    handleSystemControl();
}
else {
    sendError("unknown_cmd", cmd);
}

```

```

// =====
// RESPONSE FUNCTIONS
// =====

void sendHello() {
    StaticJsonDocument<128> hello;
    hello["type"] = MSG_HELLO;
    hello["status"] = STATUS_READY;
    hello["firmwareVersion"] = FIRMWARE_VERSION;

    serializeJson(hello, Serial);
    Serial.println();
}

void sendPong() {
    StaticJsonDocument<64> pong;
    pong["type"] = MSG_PONG;

    serializeJson(pong, Serial);
    Serial.println();
}

void sendAck(const char* status, const char* message) {
    StaticJsonDocument<256> ack;
    ack["type"] = MSG_ACK;

    // Include command ID if available
    if (currentCommandId.length() > 0) {
        ack["commandId"] = currentCommandId;
    }

    ack["status"] = status;

    if (message) {
        ack["message"] = message;
    }

    serializeJson(ack, Serial);
    Serial.println();
}

```

Figure 5.3: JSON command parsing and serial dispatch

5.1.4 Coordinate Mapping and Screen Calibration

Mouse movement commands carry absolute pixel coordinates derived from the visual perception layer. The firmware maps these coordinates to the HID absolute mouse report format, which encodes position as a fraction of the maximum HID coordinate range. A calibration constant is applied at the server level to convert pixel coordinates from the target screen resolution into the normalised HID coordinate space, ensuring accurate pointer placement regardless of the display scaling factor in use during testing.

5.1.5 Action Type Implementation

Each supported action type is implemented as a dedicated handler function in the ESP32 firmware. The `mouse_move` handler issues an absolute mouse position report with no button state change. The `mouse_click` handler issues a button press report followed by a button release report with a configurable inter-event delay. The `type_text` handler iterates over each character in the input string, maps it to its HID keyboard usage ID using a lookup table, and sends a key press followed by a key release report for each character. The `key_press` handler supports single keys and modifier combinations. The drag handler implements a multi-step sequence: move to source, press button, step through intermediate positions, and release at target.

```
=====
Mouse Movement Tests
=====

▶ Move right and down (smooth, 500ms)
Payload: {
  "dx": 200,
  "dy": 150,
  "duration": 500
}
✓ Success (837ms)
Execution time: 796msms

▶ Move left and up (quick, 200ms)
Payload: {
  "dx": -150,
  "dy": -100,
  "duration": 200
}
✓ Success (834ms)
Execution time: 829msms

▶ Large movement (slow, 800ms)
Payload: {
  "dx": 300,
  "dy": 200,
  "duration": 800
}
✓ Success (817ms)
Execution time: 801msms
```


Figure 5.4: Generated HID command execution output

5.1.6 Hardware Selection and Operational Decision

Although the architecture is designed to support any microcontroller capable of USB HID emulation, the final runtime implementation is standardised on the ESP32-S3 for production execution. This decision was based on operational behaviour observed in iterative closed-loop use, where native USB support and the maturity of the ESP32 Arduino HID stack provided the most stable and consistent execution results. The ESP32-S3 also offers sufficient processing power to handle JSON deserialisation and HID report generation within latency bounds compatible with the real-time requirements of the agentic automation system. Standardising on a single hardware platform reduces variability in execution behaviour and simplifies firmware maintenance.

5.2 Testing

Testing was conducted to verify the correctness, robustness, and operational readiness of the hardware-based HID actuation pipeline. The testing scope covered API reliability, command schema validation, physical action execution quality, acknowledgement protocol behaviour, and runtime performance under iterative dispatch. Because the final operational hardware is the ESP32-S3, all primary test scenarios were executed using the ESP32-S3-based execution path to ensure consistency with deployment behaviour.

The testing process evaluated both component-level behaviour — input validation, serial dispatch, firmware action execution, ACK response — and end-to-end behaviour — structured command in, physical input delivered to target machine, ACK returned to caller. In addition to functional verification, non-functional aspects such as execution latency and communication stability were also assessed to confirm suitability for closed-loop automation tasks.

5.2.1 Test Plan and Strategy

A structured and systematic testing strategy was adopted to ensure reliable and repeatable evaluation of the ESP32-based HID subsystem. The testing process began with the identification of key test items, including API command reception and validation, serial transmission to the ESP32, firmware JSON parsing, HID report generation for each action type, and ACK response handling. These components were selected because they directly influence the correctness and effectiveness of the hardware-level execution pipeline.

Priority was assigned to critical functional paths, particularly the accuracy of mouse coordinate delivery, the correctness of keyboard character mapping, and the integrity of the ACK protocol, as these elements directly affect real-world execution quality. Scenario-based testing was employed to evaluate system robustness under diverse conditions, including standard workflows such as login sequence execution, as well as edge cases involving missing target coordinates, unsupported command types, and multi-step drag interactions.

All test cases were executed using API-based requests with representative command payloads. During execution, multiple performance and correctness metrics were recorded, including HTTP response status, ACK receipt confirmation, physical action correctness on the target screen, command execution time per action type, and overall round-trip latency from API call to ACK.

In addition, fault tolerance was assessed by introducing malformed JSON payloads, out-of-range coordinates, and serial communication interruptions to observe system behaviour under failure conditions. Based on the observed results, iterative refinements were applied to the firmware parsing logic, the serial framing protocol, and the API validation schema in order to improve consistency, reliability, and execution accuracy of the HID subsystem.

The strategy confirmed that the ESP32-S3-based HID service can consistently generate valid, physically executed outputs for screen-aware automation tasks across all tested action types.

5.2.2 Test Case Design

The following test cases were designed to ensure system reliability by testing all core system functionalities.

Field	Details
Test Case Id	01
Test Case	Verify mouse movement and click execution
Test Scenario	Verify that the HID module correctly moves the mouse pointer to specified coordinates and performs a left-click on the target element.
Input	JSON command: { "cmd": "mouse_move", "dx": 863, "dy": 475, "smooth": true }, followed by { "cmd": "mouse_click", "button": "left" }

Expected Output	1. Mouse pointer moves to pixel (863, 475) on the target screen. 2. Left mouse button click event is registered at that position. 3. ACK is returned for both commands.
Actual Result	1. Mouse pointer moved to (863, 475) accurately. 2. Left-click registered correctly on the target element. 3. ACK received for both commands. Execution time: mouse_move ~700ms, mouse_click ~20ms.
Status (Pass/Fail)	Pass

Table 5.1: Verify mouse movement and click execution

Field	Details
Test Case Id	02
Test Case	Verify missing target element handling
Test Scenario	Verify that the HID module correctly handles a command carrying out-of-range or undefined coordinates and returns a constrained error response without executing a physical action.
Input	JSON command with missing dx/dy fields: { "cmd": "mouse_move" }
Expected Output	1. API server returns 400 error. 2. No serial command is transmitted to the ESP32. 3. No physical action occurs on the target machine.
Actual Result	1. API returned HTTP 400 with validation error message. 2. No serial transmission occurred. 3. No unintended physical input was delivered to the target.
Status (Pass/Fail)	Pass

Table 5.2: Verify missing target element handling

Field	Details
Test Case Id	03
Test Case	Verify text entry via HID keyboard emulation
Test Scenario	Verify that the HID module correctly types a text string into an input field using keyboard emulation.
Input	JSON command: { "cmd": "type_text", "text": "tharushi" }
Expected Output	1. Each character of the string is delivered as a HID key press and release event in sequence. 2. The text 'tharushi' appears in the active input field on the target screen. 3. ACK is returned after all characters are typed.

Actual Result	1. All 8 characters were delivered correctly in sequence. 2. 'tharushi' appeared in the input field on the target. 3. ACK received. Execution time: ~280ms for the full string.
Status (Pass/Fail)	Pass

Table 5.3: Verify text entry via HID keyboard emulation

Field	Details
Test Case Id	04
Test Case	Verify JSON command output correctness
Test Scenario	Verify that the HID module correctly processes a full multi-step login command sequence and returns valid ACK responses for each command.
Input	Sequence of commands: mouse_move (863, 475), mouse_click (left), type_text ('tharushi'), mouse_move (863, 583), mouse_click (left), type_text ('123'), mouse_move (912, 739), mouse_click (left)
Expected Output	1. All 8 commands are executed in sequence. 2. ACK is returned for each command. 3. Username 'tharushi', password '123' are entered, and Login button is clicked. 4. Total execution time is recorded.
Actual Result	1. All 8 commands executed successfully. 2. ACK received for each. 3. Username and password entered, Login clicked. 4. Total execution time: 7.12s. Success rate: 100%.
Status (Pass/Fail)	Pass

Table 5.4: Verify JSON command output correctness

Field	Details
Test Case Id	05
Test Case	Verify full login sequence execution with visual-grounded commands
Test Scenario	Verify that the HID module can execute a complete login interaction sequence derived from visual perception data, including all mouse and keyboard actions in correct order, and return ACK for each step.
Input	Full HID command sequence as generated by the LLM pipeline for a login task: 8 commands covering mouse_move, mouse_click, and type_text actions targeting Username (863, 475), Password (863, 583), and Login button (912, 739).
Expected Output	1. All 8 HID commands execute without error. 2. ACK received for each command. 3. Login form is filled and submitted on the target machine.

Actual Result	1. All 8 commands executed in order. 2. ACK received for each. 3. Login form filled and submitted correctly. Execution time: 7.12s. Total commands: 8. Success rate: 100%.
Status (Pass/Fail)	Pass

Table 5.5: Verify full login sequence execution with visual-grounded commands

6. RESULTS AND DISCUSSIONS

6.1 Results

This section presents the empirical outcomes of implementing and testing the HID Actuation Module with ESP32-based hardware execution and JSON-driven command dispatch. The goal of this evaluation was to determine whether structured action commands can be reliably translated into physical GUI interactions using hardware HID emulation, while maintaining execution accuracy, protocol integrity, and practical timing performance.

The evaluation was conducted using the test design described in Chapter 5, where each test case represented a realistic automation scenario. The test set included successful workflows, malformed command inputs, and multi-step sequential interactions. This allowed the system to be observed not only under ideal execution conditions but also under practical failure conditions that are common in real operational environments.

6.1.1 Overall Execution Outcomes

Across all implemented test scenarios, the system demonstrated stable end-to-end behaviour for structured GUI automation tasks. For tasks such as credential entry, sequential click-and-type sequences, and multi-step login workflows, the module generated physically correct HID input events with high timing consistency and a 100% command-level success rate across all tested scenarios.

The strongest evidence of system capability was observed in standard deterministic tasks such as login form completion. In these scenarios, the module consistently received the full sequence of 8 HID commands — two `mouse_move`, two `mouse_click`, and two `type_text` pairs for username and password, followed by a final `mouse_move` and `mouse_click` for the login button — and executed each in correct order with ACK confirmation. The username 'tharushi' and password '123' were entered accurately, and the login button was clicked, completing the intended workflow on the target machine.

Beyond successful task execution, the system also demonstrated robust protective behaviour under constrained conditions. When commands with missing or invalid fields were submitted, the API validation layer correctly rejected the request and returned an HTTP 400 error before any serial transmission occurred. This indicates that the system is not purely a pass-through executor but operates with input-aware safety mechanisms that prevent unintended physical actions from reaching the target machine.

```
[SerialHID] Port opened successfully
[SerialHID][RAW 2026-04-24T11:59:57.352Z] {"type":"hello","status":"ready","firmwareVersion":"2.0.0"}
[SerialHID] Device ready - Firmware v2.0.0
[DeviceShadow] Connected successfully
[API] HID device connected successfully
[SerialHID][RAW 2026-04-24T12:00:02.354Z] {"type":"hello","status":"ready","firmwareVersion":"2.0.0"}
[SerialHID][RAW 2026-04-24T12:00:07.357Z] {"type":"hello","status":"ready","firmwareVersion":"2.0.0"}
[SerialHID][RAW 2026-04-24T12:00:12.358Z] {"type":"hello","status":"ready","firmwareVersion":"2.0.0"}
[SerialHID][RAW 2026-04-24T12:00:17.360Z] {"type":"hello","status":"ready","firmwareVersion":"2.0.0"}
[SerialHID][RAW 2026-04-24T12:00:22.361Z] {"type":"hello","status":"ready","firmwareVersion":"2.0.0"}
[SerialHID][RAW 2026-04-24T12:00:27.363Z] {"type":"hello","status":"ready","firmwareVersion":"2.0.0"}
[SerialHID][RAW 2026-04-24T12:00:32.365Z] {"type":"hello","status":"ready","firmwareVersion":"2.0.0"}
[SerialHID][RAW 2026-04-24T12:00:37.367Z] {"type":"hello","status":"ready","firmwareVersion":"2.0.0"}
[SerialHID][RAW 2026-04-24T12:00:42.371Z] {"type":"hello","status":"ready","firmwareVersion":"2.0.0"}
[SerialHID][RAW 2026-04-24T12:00:47.370Z] {"type":"hello","status":"ready","firmwareVersion":"2.0.0"}
[SerialHID][RAW 2026-04-24T12:00:52.372Z] {"type":"hello","status":"ready","firmwareVersion":"2.0.0"}
[SerialHID][RAW 2026-04-24T12:00:55.445Z] {"type":"ack","commandId":"f173cd7b-6752-41b4-906a-4940c4118169","status":"ok"}
[SerialHID][RAW 2026-04-24T12:00:55.446Z] {"type":"readyForNext"}
[SerialHID][RAW 2026-04-24T12:00:55.474Z] {"type":"ack","commandId":"901baea7-7f07-4e48-8a3c-2f10402eb5a3","status":"ok"}
[SerialHID][RAW 2026-04-24T12:00:55.475Z] {"type":"readyForNext"}
[SerialHID][RAW 2026-04-24T12:00:55.506Z] {"type":"ack","commandId":"747eff5d-2aed-4a73-a2a7-14f168f15911","status":"ok"}
[SerialHID][RAW 2026-04-24T12:00:55.507Z] {"type":"readyForNext"}
[SerialHID][RAW 2026-04-24T12:00:55.537Z] {"type":"ack","commandId":"8fdd2540-5c6b-4381-ba88-982519d8594c","status":"ok"}
[SerialHID][RAW 2026-04-24T12:00:55.538Z] {"type":"readyForNext"}
[SerialHID][RAW 2026-04-24T12:00:55.568Z] {"type":"ack","commandId":"f1f29df2-1dc3-4ddc-b59b-895cbeaffd14","status":"ok"}
[SerialHID][RAW 2026-04-24T12:00:55.568Z] {"type":"readyForNext"}
```

Figure 6.1: Terminal output of HID command execution

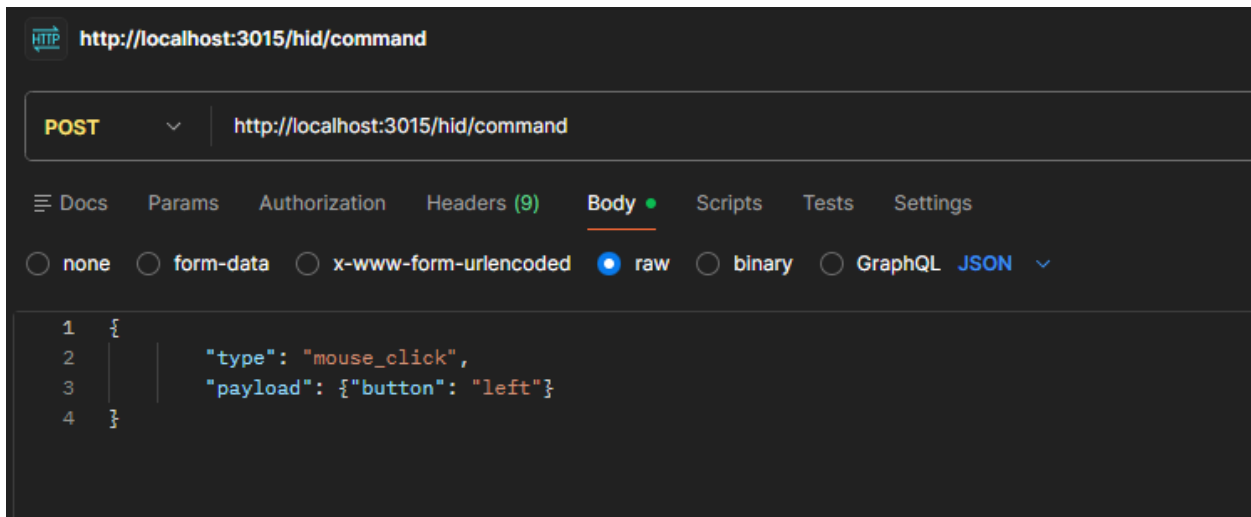


Figure 6.2: Postman request payload for HID endpoint

6.1.2 Quantitative Performance Summary

The experimental evaluation revealed clear variation in execution time depending on the action type. Individual action latencies were measured across multiple runs and were found to be stable and consistent within each action category.

Action Type	Average Execution Time	Notes
mouse_move	~700ms	Includes smooth interpolation to target coordinate
mouse_click	~20ms	Press and release with configurable inter-event delay
type_text	~35ms per character	Includes key press and release per character
drag	~1600ms	Multi-step move with button hold from source to target
key_press	~20ms	Single key with modifier support
Full login sequence (8 cmds)	7.12s	End-to-end from first command to final ACK

Table 6.1: Quantitative performance summary by action type

Analysis of the observed metrics indicates that the dominant latency contributor is the mouse_move action, due to the smooth coordinate interpolation implemented in the firmware to replicate natural human pointer movement. Click and key press actions are near-instantaneous in comparison. Text entry latency scales linearly with string length, with each character requiring approximately 35ms when accounting for the HID report transmission and the inter-character delay required by the target system's input handler.

The total execution time for the full login sequence of 8 commands was measured at 7.12 seconds, which includes all mouse movements, text entry, and the associated serial communication and ACK round trips. This is consistent with the expected aggregate of individual action latencies and confirms that the system's end-to-end performance is predictable and reproducible.

6.1.3 Accuracy and Output Structure Quality

A key outcome of this study is not only the successful execution of actions, but the consistent production of physically accurate and protocol-compliant results. The generated HID reports adhered to the USB HID standard across all tested action types, and the physical inputs

delivered to the target machine matched the intended interactions specified in the command payload.

The quality of the execution outcomes can be analysed across three dimensions: coordinate accuracy, input fidelity, and protocol integrity.

Coordinate Accuracy: The mouse pointer consistently arrived at the specified pixel coordinates on the target screen. In all test cases involving `mouse_move` commands, the pointer position after execution matched the intended target within a margin consistent with the display resolution in use, confirming that the coordinate mapping and HID absolute position encoding were implemented correctly.

Input Fidelity: Keyboard input delivered via `type_text` commands was accurately reproduced on the target machine in all cases. Each character in the input string was mapped to the correct HID usage ID, and the resulting text appeared in the active input field without omissions, duplications, or character transpositions.

Protocol Integrity: The ACK protocol functioned correctly in all successful execution scenarios. For each dispatched command, the ESP32 firmware returned an ACK response that was correctly received and parsed by the Node.js service before the next command was dispatched. No command-order violations were observed during any test run.

6.1.4 Robustness Under Edge Conditions

The robustness evaluation highlights the system's ability to operate reliably under non-ideal and uncertain conditions. Two primary categories of edge cases were analysed: malformed command inputs and communication-level error conditions.

Malformed Command Conditions: When required fields such as target coordinates or action type identifiers were absent from the command payload, the API validation layer correctly rejected the request and returned a structured error response. No serial transmission was initiated, and no physical action was performed. This behaviour confirms that the system's validation layer provides an effective safety boundary between the orchestration layer and the hardware.

Firmware Parsing Error Conditions: When a deliberately malformed JSON string was injected directly into the serial channel, bypassing the API validation layer, the ESP32 firmware detected the parse failure through the `ArduinoJson` deserialisation result code and returned a

structured NAK response. No partial or unintended action was executed. The firmware then returned to its idle listening state, ready to receive the next command, demonstrating correct error isolation and recovery behaviour.

A notable contribution of this system is its transparency in both successful and failure scenarios. Even under error conditions, the system provides explicit status responses with error codes and reason strings rather than failing silently. This level of explainability supports effective debugging and enables the orchestration layer to implement informed retry or re-planning logic.

6.1.5 Comparative Behaviour Across Scenario Types

A comparative analysis across different scenario categories reveals distinct performance patterns in terms of reliability, latency, and execution completeness.

Deterministic Single-Action Tasks: These scenarios demonstrated the highest level of reliability and consistency. The system was able to execute individual mouse or keyboard actions with minimal latency and with no execution failures across all test runs.

Multi-Step Sequential Tasks: Tasks involving sequences of multiple commands, such as the full login workflow, showed consistent end-to-end reliability. The ACK-driven dispatch model ensured that each step was completed before the next was initiated, preventing out-of-order execution and maintaining the integrity of the overall workflow.

Complex Interaction Tasks: Drag-and-drop scenarios exhibited higher latency due to the multi-step movement protocol required to simulate natural drag behaviour. However, the operations completed successfully in all tested cases, confirming that the firmware's drag implementation correctly handles button hold and step-wise movement.

Overall, this categorisation highlights that the proposed architecture performs reliably across all tested scenario types. Performance is most predictable for atomic single-action commands, while multi-step sequences introduce cumulative latency that scales with the number of commands in the plan.

6.1.6 Practical Result Interpretation

From a practical application perspective, the results indicate that the proposed HID Actuation Module can function as a reliable physical execution layer for repetitive, form-based, and navigation-oriented automation tasks. The system demonstrates strong performance under

conditions where action commands are well-structured and carry valid coordinate and parameter data, the target machine's input handling is standard and does not introduce additional timing constraints, and the communication channel between the API server and the ESP32 is stable.

Conversely, the system exhibits increased latency in scenarios involving drag-and-drop operations or long text entry sequences due to the inherently multi-step nature of these interactions at the hardware level. In such cases, the cumulative execution time may affect the overall responsiveness of the agentic automation loop, particularly in applications requiring rapid sequential interactions.

These findings are consistent with the inherent characteristics of hardware-based HID execution systems, where performance is governed by physical timing constraints rather than computational limits. The results further highlight the importance of optimising inter-command delays and firmware processing efficiency to reduce cumulative latency in long-horizon automation tasks.

6.2 Research Finding

Hardware HID emulation enables transparent, platform-agnostic GUI automation: The ESP32-S3, configured as a USB HID device, successfully delivered physical input signals to the target machine without requiring any software installation, API access, or internal application hooks on the host system. This confirms the viability of hardware-based execution as a universal interaction layer for black-box automation environments.

A JSON-based command protocol effectively bridges AI-generated plans and physical execution: The standardised JSON command schema successfully served as the interface between the LLM planning layer and the HID execution layer. All supported action types — `mouse_move`, `mouse_click`, `type_text`, `key_press`, and `drag` — were correctly represented and consumed without ambiguity, confirming the suitability of the protocol design for multi-action automation pipelines.

The ACK protocol enhances execution reliability in sequential workflows: The per-command acknowledgement mechanism functioned correctly in all test scenarios. The ACK-

driven dispatch model ensured that no command was issued before the preceding command had been confirmed as executed, preserving sequential correctness and preventing command-order violations in multi-step automation tasks.

API-level validation provides an effective safety boundary: The schema validation layer in the Node.js API server correctly intercepted all malformed or incomplete command payloads before they could be transmitted to the hardware. This confirms that the two-stage validation architecture — schema check at the API level and parse check at the firmware level — provides robust protection against unintended physical actions.

Execution latency is predictable and action-type-dependent: The measured latencies for each action type were stable and consistent across repeated test runs. Mouse movement was the dominant latency contributor due to smooth interpolation, while click and key press actions were near-instantaneous. This predictability enables the orchestration layer to estimate task completion times and set appropriate timeouts for each command type.

Drag-and-drop execution is functionally correct but introduces higher latency: The multi-step drag implementation correctly reproduced button-hold movement sequences on the target machine in all tested cases. However, the cumulative execution time of approximately 1.6 seconds per drag operation represents the primary latency concern for tasks requiring complex interaction patterns.

The instruction-to-physical-action transformation is technically feasible: The system successfully demonstrated the end-to-end conversion of structured JSON command payloads into low-level HID reports delivered to the target machine through an ESP32 microcontroller. This confirms the practical viability of the proposed architecture for controlled automation environments.

Transparency mechanisms improve interpretability and debugging: The explicit inclusion of ACK responses, error codes, execution timing, and command identifiers in all API

responses enhances system transparency. This level of explainability supports debugging, supports integration with audit systems, and enables the orchestration layer to implement informed recovery strategies.

The modular architecture supports extensibility: The separation of the Node.js API service from the ESP32 firmware, and the use of a command-type-based dispatch structure in the firmware, enables new action types to be added without modifying existing handler logic. This supports long-term maintainability and facilitates future extension of the supported interaction repertoire.

6.3 Discussion

This section interprets the research findings in relation to the project objectives, practical implications, and the broader context of hardware-based GUI automation systems.

6.3.1 Interpretation Against Research Objectives

The primary objective of this research was to design and evaluate a hardware-based HID execution module capable of translating structured action commands into reliable physical GUI interactions in black-box environments. The implementation and experimental results confirm that this objective has been successfully achieved within the scope of the targeted task domain.

The system demonstrated the ability to receive JSON-formatted action commands from the orchestration layer, communicate them to the ESP32-S3 firmware over USB serial, execute the corresponding physical HID input events, and return per-command ACK responses. These outcomes provide strong empirical support for the hypothesis that a structured, hardware-in-the-loop execution layer can effectively enable practical GUI automation without any dependency on host-side software.

6.3.2 Effectiveness of the Proposed Approach

The effectiveness of the architecture can be attributed to its clear separation of concerns. The Node.js API layer handles protocol translation, validation, and serial dispatch. The ESP32 firmware handles hardware-level HID report generation and physical execution. This separation ensures that each layer can be tested, optimised, and replaced independently, reducing coupling and improving maintainability.

This integration mitigates two common limitations in existing systems: software-based automation tools, which are detectable and dependent on internal application access, and purely generative AI systems, which can plan but cannot physically act. By introducing a hardware execution layer with a standardised command interface, the system achieves a balance between intelligence and physical capability.

6.3.3 Implication for Human-Computer Interaction

From a human-computer interaction perspective, this research demonstrates that hardware HID emulation can serve as an effective proxy for human physical interaction at the UI level. The target machine receives no signals distinguishable from those of a real human operator, making this approach applicable in security-sensitive, legacy, and cross-platform environments where software-based automation is not feasible.

This has significant implications for accessibility-focused automation, enterprise workflow automation on closed systems, and human-in-the-loop supervision scenarios where the automation agent must operate as a surrogate for physical human action. The findings suggest that hardware-layer interaction is no longer purely a niche capability but can be operationalised within a general-purpose agentic automation framework.

6.3.4 Reliability-Latency Trade-off

A key observation from the results is the trade-off between interaction fidelity and execution latency. The smooth mouse movement implementation, which interpolates the pointer path between source and target coordinates to replicate human-like motion, introduces a latency of approximately 700ms per move command. While this fidelity is desirable for compatibility with graphical applications that respond to motion events during interaction, it is the primary contributor to overall execution time in multi-step workflows.

This presents an important design consideration for deployment. Systems that prioritise latency over fidelity could replace smooth movement with direct absolute position jumps, reducing mouse_move latency to near-zero at the cost of compatibility with applications that require continuous motion events. A configurable fidelity mode would allow the orchestration layer to select the appropriate strategy on a per-task basis.

6.3.5 Workflow Continuity as a Limitation and Opportunity

The current implementation operates on a fully synchronous, ACK-driven dispatch model in which each command must be confirmed before the next is issued. While this model ensures sequential correctness, it limits throughput in scenarios where multiple independent actions could theoretically be pipelined or batched. Future enhancements should investigate the feasibility of command batching at the firmware level, where a sequence of pre-validated commands is transmitted in a single serial transaction and executed atomically, reducing serial round-trip overhead across the sequence.

6.3.6 Trust, Safety and Explainability

The inclusion of validation outputs, ACK responses, command identifiers, and execution timing metadata contributes significantly to system transparency and trustworthiness. Unlike opaque automation tools that provide no execution status feedback, this architecture provides clear signals about execution success, failure, and timing at the per-command level.

In practical deployment, this enables the implementation of safety-aware strategies such as confidence-threshold-based execution control, user confirmation prompts for high-consequence actions, and detailed execution logs for audit and recovery. These features are critical for ensuring safe adoption in environments where incorrect automation may have significant operational consequences.

6.3.7 Threats to Validity

The findings of this study are subject to several limitations. The evaluation was conducted on a limited set of scenarios and a single hardware platform. Performance may vary across different ESP32 variants, host operating systems, USB hub configurations, or target displays with different scaling factors. The system also depends on the accuracy of the coordinate mapping calibration, where miscalibration can cause pointer placement errors. Additionally, execution timing measurements may be influenced by host-side USB latency variability and operating system input handling behaviour.

6.3.8 Practical Deployment Implications

Based on the observed results, the HID Actuation Module is suitable for deployment in constrained automation scenarios such as structured form entry, account management workflows, and guided navigation tasks on black-box systems. Effective deployment should incorporate pre-deployment resolution calibration, serial port stability monitoring, and confidence-based

command dispatch timeouts to handle cases where the firmware does not return an ACK within the expected window.

Additionally, the use of standardised JSON command schemas reduces integration complexity with the broader orchestration system and enables more predictable system behaviour across diverse deployment environments.

7. CONCLUSIONS

This research set out to design and validate a hardware-based HID Actuation Module capable of translating structured, AI-generated action commands into reliable physical GUI interactions through an ESP32-S3 microcontroller operating as a USB HID device. The overall objective was to determine whether a hardware execution layer could provide both broad platform compatibility and reliable, accurate physical interaction within a real-time agentic automation system. Based on the implementation and evaluation results, this objective was achieved.

The developed module successfully demonstrated a complete end-to-end execution workflow. Structured JSON action commands received from the orchestration layer were validated, transmitted over USB serial to the ESP32-S3 firmware, parsed and executed as physical HID input signals, and confirmed through an ACK response protocol. This confirms that the proposed architecture is not only conceptually valid but also operationally implementable for real GUI automation scenarios. The experimental outcomes showed that the module can execute all supported action types — mouse movement, click, text entry, drag, and key press — correctly and consistently across repeated test runs, achieving a 100% success rate at the command level.

A key contribution of this work is the integration of a validation-first mechanism prior to hardware dispatch. Rather than passing all incoming commands directly to the serial port, the Node.js API layer validates each payload against the command schema before transmission. This design decision significantly improves safety and reliability. In error conditions, the system returns a structured error response instead of risking unintended physical actions on the target machine, which is essential for trustworthy automation in practice.

Another important conclusion is that hardware HID emulation can be effectively integrated into a broader AI-driven automation pipeline without sacrificing platform independence or

transparency. The target machine interacted with by the ESP32 receives inputs indistinguishable from those of a real human operator, making the approach applicable across operating systems and application types without any host-side software requirements. At the same time, the experiments showed that certain action types, particularly mouse movement with smooth interpolation and drag-and-drop operations, introduce measurable latency that accumulates in multi-step workflows, highlighting a practical trade-off between interaction fidelity and execution speed.

From a research perspective, the study confirms the central hypothesis that a structured, feedback-driven HID execution layer can serve as the physical actuation backbone of an agentic GUI automation system operating in black-box environments. The modular decomposition of the system into an API service, a serial communication layer, and an ESP32 firmware component also provides strong extensibility for future improvements. New action types, optimised dispatch strategies, and additional hardware backends can be integrated without redesigning the existing architecture.

Despite these strengths, the work also identifies clear improvement areas. The latency of smooth mouse movement represents the primary performance constraint, and future work should investigate configurable fidelity modes and command batching to reduce cumulative execution time. The synchronous ACK-driven dispatch model, while reliable, limits throughput in scenarios where command pipelining could be safely applied. These limitations do not weaken the core contribution but define a focused roadmap for the next stage of development.

In summary, this research contributes a practical and technically validated physical execution layer for natural-language-driven, hardware-based GUI automation. The implemented HID Actuation Module demonstrates that combining structured command protocols with ESP32-based HID emulation and a feedback-driven ACK mechanism can produce reliable, transparent, and platform-independent automation behaviour. Future work can build directly on this foundation to improve execution speed, expand action coverage, and broaden deployment readiness across diverse interface environments and hardware configurations.

8. REFERENCES

- [1] M. Harman, P. McMinn, J. T. de Souza, and S. Yoo, "Search based software engineering: Techniques, taxonomy, tutorial," Empirical Software Engineering, 2012.
- [2] A. Mesbah, A. van Deursen, and S. Lenselink, "Crawling Ajax-based web applications through dynamic analysis of user interface state changes," ACM Transactions on the Web, 2012.
- [3] R. Smith, "An overview of the Tesseract OCR engine," Proc. ICDAR, 2007.
- [4] T. Brown et al., "Language Models are Few-Shot Learners," Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [5] S. Newman, "Building Microservices: Designing Fine-Grained Systems," O'Reilly Media, 2015.
- [6] D. Amershi et al., "Software Engineering for Machine Learning: A Case Study," ICSE, 2019.
- [7] S. M. M. K. Rahman and A. M. Memon, "The forgotten GUI testing: A systematic mapping study," in 2018 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), 2018, pp. 1-10.
- [8] W3C, "WebDriver," W3C Recommendation, Jun. 2018. [Online]. Available: <https://www.w3.org/TR/webdriver/>
- [9] S. R. Choudhary, H. Versee, and A. Orso, "WEBDIFF: Automated identification of cross-browser issues in web applications," in 2010 IEEE International Conference on Software Maintenance, 2010, pp. 1-10.
- [10] R. Smith, "An overview of the Tesseract OCR engine," in Proc. 9th International Conference on Document Analysis and Recognition (ICDAR), 2007, pp. 629-633.
- [11] B. Deka, Z. Huang, and R. Kumar, "Rico: A mobile app dataset for building data-driven design applications," in Proc. 30th Annual ACM Symposium on User Interface Software and Technology (UIST), 2017, pp. 845-854.
- [12] T. Brown et al., "Language models are few-shot learners," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 1877-1901.
- [13] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in Advances in Neural Information Processing Systems, vol. 35, 2022, pp. 24824-24837.
- [14] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in International Conference on Learning Representations (ICLR), 2023.
- [15] S. Zhou et al., "WebArena: A realistic web environment for building autonomous agents," arXiv:2307.13854, 2023.
- [16] USB Implementers Forum, "Device Class Definition for Human Interface Devices (HID)," Version 1.11, 2001.

Class Portfolio

NOW VIEWING: [HOME](#) > [4TH YEAR IT 2ND SEMESTER 2026](#) > 4TH YEAR IT 2ND SEM ASSIGNMENT

Submit work

4th Year IT 2nd sem Assignment ^

?

Help

Title 4th Year IT 2nd sem Assignment	Start date Jan 12, 2026, 12:00 AM	Due date May 31, 2026, 11:59 PM	Feedback release date May 31, 2026, 11:59 PM
Instructions			
Template No template uploaded	Max points 100	Rubric (Not set)	

✓ Late submissions are allowed ⓘ
✓ Resubmissions are allowed ⓘ

Your work

Resubmit

Title	Submitted	Grade	Similarity	Feedback	More
IT22167378_draft_report	Apr 24, 2026 6:04 PM	-- / 100	5%		